DATA-260 Homework 8 - Kafka Request/Response, Agent Pipeline and Failure Behaviour

Pinal Pawar · SID4 7875 · **GitHub:** https://github.com/PinalPawar/data260-7875 (tag `hw8`; collaborators Sbnikitha and supriyaselvanganesan have access)

Section 0 - Personal Configuration and Domain

Value	This submission
SID4	7875
PORT_BASE = 8000 + (SID4 mod 900)	8675
PREFIX = "s" + SID4	s7875
SEED = SID4	7875
VERIFY_SEED = 260000 + SID4	267875
DOMAIN_ID = SID4 mod 8	3 - Grocery supply and recall notices (primary entity: Notice)
Hardware	MacBook Air, Apple M2, 16 GB RAM; Docker Desktop
Local model	qwen3:1.7b via Ollama (temperature 0, seed 7875, thinking off)
Kafka	apache/kafka:3.9.2, single node, KRaft (no ZooKeeper)
Tagged commit	Code + results commit `58c8954` (the commit `verification.json` was run on). Tag `hw8` is on the commit right after it, which only adds this report and the re-run verification.json / RUN_LOG.txt.

Repository additions (all in the shared root folders, extending HW1-HW7): `kafka_hw8/` (worker, FastAPI wrapper, Kafka client, shared agent loop), `homework_demo.py`, `planner.py`, `writer.py`, `reviewer.py`, `send_question.py`, `docker-compose.kafka.yml`, `Dockerfile.kafka`, `scripts/hw08_create_topics.sh`, `scripts/run_hw08_part3.py`, `scripts/verify_hw08.py`, `tests/test_hw08_kafka.py`, and `reports/hw08/` (RUN_LOG.txt, raw/, METRICS.md, AI_USE.md, verification.json, report.pdf). Notice validation reuses the existing `NoticeCreate` schema from `schemas.py` (HW5).

Part 1 - Backend Service Request/Response over Kafka

1.1 Topics and message schemas

ENTITY is replaced by the primary entity **Notice**: `CREATE_NOTICE`, `GET_NOTICE`, `notice_id`. The same rule is applied to the two topics, so `<PREFIX>_entity_requests` becomes `s7875_notice_requests` and `<PREFIX>_entity_responses` becomes `s7875_notice_responses` (one partition each). These are the only two topics the service uses.

Request schema (s7875_notice_requests) - from the 11:46:53 run


```
{"correlation_id": "007c1cf0-c86c-4024-a84d-fa942ebc9c5f",
 "reply_to": "s7875_notice_responses",
 "operation": "CREATE_NOTICE",
 "data": {"product": "Organic Baby Spinach 5oz", "notice_code": "RCL-2026-07875",
          "units_affected": 4471, "manufacturer_id": 1,
          "email": "recalls@greenfields-foods.com",
          "description": "Routine testing found Listeria monocytogenes in lot L4471. ...",
          "category": "bacterialContamination"}}
```


Response schema (s7875_notice_responses) - success and error; every response echoes the correlation_id


```
CREATE ok : {"correlation_id": "007c1cf0-...", "operation": "CREATE_NOTICE", "success": true,
             "notice_id": "9264c6ab-d48c-4268-9db1-ee6113f956df", "message": "Notice created"}
GET ok    : {"correlation_id": "233d96f0-...", "operation": "GET_NOTICE", "success": true,
             "notice": {"notice_id": "9264c6ab-...", "product": "Organic Baby Spinach 5oz", ...}}
error     : {"correlation_id": "c6668ec4-...", "operation": "GET_NOTICE", "success": false,
             "error": "Notice not found"}
```


correlation_id flow. The client creates the id once per request (step 1 / step 8), puts it in the envelope, and passes it to the API wrapper unchanged when the API is used. The worker copies it from the request into the response, success or error, and the client matches the reply by that id. CREATE and GET each get their own id: two requests with one id would be indistinguishable to the client and to the worker's duplicate check (Part 3), so "create one correlation_id" is applied per request.

1.2 Code: worker, validation, Kafka client, API wrapper

kafka_hw8/notice_service.py (lines 75-98) - validation + plain dict store + UUID

```
    def _process(self, request) -> dict:
        if not isinstance(request, dict):
            return {"success": False, "error": "Request must be a JSON object"}
        envelope = ("correlation_id", "reply_to", "operation", "data")
        missing = [k for k in envelope if k not in request]
        if missing:
            return {"success": False, "error": f"Malformed request, missing: {', '.join(missing)}"}
        op = request["operation"]
        data = request["data"]
        if op == OP_CREATE:
            clean, error = validate_notice(data)
            if error:
                return {"success": False, "error": error}
            notice_id = str(uuid.uuid4())
            self.notices[notice_id] = {ID_FIELD: notice_id, **clean}
            return {"success": True, ID_FIELD: notice_id, "message": "Notice created"}
        if op == OP_GET:
            notice_id = data.get(ID_FIELD) if isinstance(data, dict) else None
            if not isinstance(notice_id, str) or not notice_id.strip():
                return {"success": False, "error": f"{ID_FIELD} is required"}
            if notice_id not in self.notices:
                return {"success": False, "error": "Notice not found"}
            return {"success": True, "notice": self.notices[notice_id]}
        return {"success": False, "error": "Unknown operation"}
```

kafka_hw8/notice_service.py (lines 27-49) - required fields, empty values, NoticeCreate (strict)

```
def validate_notice(data) -> tuple[dict | None, str | None]:
    if not isinstance(data, dict):
        return None, "data must be a JSON object"
    missing = [f for f in REQUIRED_FIELDS if f not in data or data[f] is None]
    if missing:
        return None, f"Missing required field(s): {', '.join(missing)}"
    empty = [f for f in REQUIRED_FIELDS if isinstance(data[f], str) and not data[f].strip()]
    if empty:
        return None, f"Empty value for field(s): {', '.join(empty)}"
    unknown = sorted(set(data) - set(REQUIRED_FIELDS))
    if unknown:
        return None, f"Unknown field(s): {', '.join(unknown)}"
    try:
        # strict: "5" is not accepted as a number and true is not accepted as 1
        notice = NoticeCreate.model_validate(data, strict=True)
    except ValidationError as exc:
        problems = []
        for err in exc.errors():
            field = ".".join(str(p) for p in err["loc"]) or "data"
            problems.append(f"{field}: {err['msg']}")
        return None, "Invalid value(s): " + "; ".join(problems)
    return notice.model_dump(), None
```

kafka_hw8/worker.py (lines 52-75) - consume -> process -> reply to reply_to -> commit

```
        async for record in consumer:
            try:
                request = json.loads(record.value.decode())
            except (UnicodeDecodeError, json.JSONDecodeError):
                request = None
            corr = request.get("correlation_id") if isinstance(request, dict) else None
            log(TAG, "RECEIVE", corr, request, topic=record.topic, p=record.partition,
                off=record.offset)

            response, duplicate = service.handle(request)
            if duplicate:
                log(TAG, "DUPLICATE", corr, note="correlation_id already answered; "
                    "re-sending stored response, nothing re-processed")
            else:
                log(TAG, "PROCESS", corr, operation=response.get("operation"),
                    success=response["success"], notices_in_store=len(service.notices))

            reply_topic = pick_reply_topic(request)
            log(TAG, "REPLY", corr, response, topic=reply_topic)
            await producer.send_and_wait(reply_topic, json.dumps(response).encode(),
                                         key=(corr or "").encode() or None)
            # commit only after the reply is safely written
            await consumer.commit({TopicPartition(record.topic, record.partition):
                                   record.offset + 1})
```

kafka_hw8/rpc_client.py (lines 26-32) - request envelope

```
def build_request(correlation_id: str, operation: str, data) -> dict:
    return {
        "correlation_id": correlation_id,
        "reply_to": RESPONSE_TOPIC,
        "operation": operation,
        "data": data,
    }
```

kafka_hw8/rpc_client.py (lines 83-101) - client: send, wait for the same id

```
    async def request(self, operation: str, data, correlation_id: str | None = None) -> dict:
        corr = correlation_id or new_correlation_id()
        message = build_request(corr, operation, data)
        future = asyncio.get_running_loop().create_future()
        self.pending[corr] = future
        # logged before the write, so a merged log never shows the worker's RECEIVE first
        log(self.tag, "SEND", corr, message, topic=REQUEST_TOPIC)
        await self.producer.send_and_wait(REQUEST_TOPIC, json.dumps(message).encode(),
                                          key=corr.encode())
        try:
            response = await asyncio.wait_for(future, timeout=self.timeout_s)
        except asyncio.TimeoutError:
            self.pending.pop(corr, None)
            response = {"correlation_id": corr, "operation": operation, "success": False,
            log(self.tag, "TIMEOUT", corr)
        if response.get("correlation_id") != corr:  # cannot happen by construction; be loud
        return response
```

kafka_hw8/api.py (lines 46-52) - FastAPI wrapper keeps a caller-supplied correlation_id

```
@app.post("/kafka/notices")
async def notice_request(body: dict = Body(...)):
    corr = body.get("correlation_id") or new_correlation_id()
    log(TAG, "HTTP-REQUEST", corr, body)
    response = await rpc.request(body.get("operation"), body.get("data"), correlation_id=corr)
    log(TAG, "HTTP-RESPONSE", corr, response)
    return response
```

1.3 Containers running (Kafka, worker, FastAPI) and topic creation

docker-compose.kafka.yml (lines 56-79) - worker and API services (Kafka service above them)

```yaml
  notice-worker:
    build:
      context: .
      dockerfile: Dockerfile.kafka
    container_name: s7875_notice_worker
    command: ["python", "-m", "kafka_hw8.worker"]
    environment:
      KAFKA_BOOTSTRAP: kafka:19092
      IDEMPOTENCY: "1"
    depends_on:
      kafka-init:
        condition: service_completed_successfully
    restart: unless-stopped

  notice-api:
    build:
      context: .
      dockerfile: Dockerfile.kafka
    container_name: s7875_notice_api
    command: ["uvicorn", "kafka_hw8.api:app", "--host", "0.0.0.0", "--port", "8675"]
    environment:
      KAFKA_BOOTSTRAP: kafka:19092
    ports:
      - "8675:8675"
```

```
CONTAINER ID   IMAGE                    COMMAND                  CREATED             STATUS                       PORTS                                                  NAMES
3ee55b4a875f   s7875_hw8-notice-api     "uvicorn kafka_hw8.a…"   About an hour ago   Up About an hour             0.0.0.0:8675->8675/tcp, [::]:8675->8675/tcp            s7875_notice_api
81fd9a16bd9b   s7875_hw8-notice-worker  "python -m kafka_hw8…"   About an hour ago   Up About an hour                                                                    s7875_notice_worker
f531514376bb   apache/kafka:3.9.2       "/__cacert_entrypoin…"   About an hour ago   Up About an hour (healthy)   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp            s7875_kafka
c1e1552c032a   mongo:7.0                "docker-entrypoint.s…"   11 days ago         Up 12 hours                  0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp        s7875_mongo
(data260) Pinal Pawar ~/Desktop/data260-7875 %
```

docker ps --filter name=s7875_ : s7875_kafka (healthy), s7875_notice_worker, s7875_notice_api on 8675 (s7875_mongo is the HW6 database).

```
2026-10-27T15:46:26.877+00:00 [WORKER] consuming s7875_notice_requests, default reply topic s7875_notice_responses
Consumer is ready and listening...
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | notice worker =====
2026-10-27T16:57:59.856+00:00 [WORKER] bootstrap=kafka:19092 group=s7875_notice_worker idempotency=True
2026-10-27T16:57:59.856+00:00 [WORKER] consuming s7875_notice_requests, default reply topic s7875_notice_responses
Consumer is ready and listening...
(data260) Pinal Pawar ~/Desktop/data260-7875 %
```

Worker container log after a restart: "Consumer is ready and listening..." (times are the container's UTC clock).

```
INFO:     Started server process [1]
INFO:     Waiting for application startup.
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | FastAPI Kafka wrapper =====
2026-10-27T15:46:27.295+00:00 [API] listening for responses on s7875_notice_responses
2026-10-27T15:46:27.295+00:00 [API] requests -> s7875_notice_requests, responses <- s7875_notice_responses
2026-10-27T15:46:27.295+00:00 [API] FastAPI server started at http://127.0.0.1:8675
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8675 (Press CTRL+C to quit)
INFO:     192.168.65.1:16856 - "GET / HTTP/1.1" 404 Not Found
INFO:     192.168.65.1:16856 - "GET /favicon.ico HTTP/1.1" 404 Not Found
(data260) Pinal Pawar ~/Desktop/data260-7875 %
```

FastAPI container log: "FastAPI server started at http://127.0.0.1:8675". The two 404 lines are a browser request for "/", which has no route.

1.4 homework_demo.py: CREATE_NOTICE then GET_NOTICE

homework_demo.py (lines 104-130) - steps 1-8

```
        step(1, "create one correlation_id for the CREATE_NOTICE request")
        create_corr = new_correlation_id()
        print(f"correlation_id = {create_corr}")

        step(2, f"send CREATE_NOTICE to {REQUEST_TOPIC}")
        step(3, "the request message (as written to the request topic)")
        print(json.dumps(build_request(create_corr, OP_CREATE, SAMPLE_NOTICE), indent=2))
        step(4, "worker receives and processes it   (see merged log in step 9)")
        step(5, f"worker replies on {RESPONSE_TOPIC}   (see merged log in step 9)")
        create_resp = await send(client, args.via, OP_CREATE, SAMPLE_NOTICE, create_corr)

        step(6, "response received by the client")
        print(json.dumps(create_resp, indent=2))
        if not create_resp.get("success"):
            raise SystemExit("CREATE_NOTICE failed - is the worker running? (docker ps)")

        step(7, f"extract the generated {ID_FIELD}")
        notice_id = create_resp[ID_FIELD]
        print(f"{ID_FIELD} = {notice_id}")

        step(8, f"send GET_NOTICE for that {ID_FIELD} (new correlation_id for this request)")
        get_corr = new_correlation_id()
        get_data = {ID_FIELD: notice_id}
        print(f"correlation_id = {get_corr}")
        print(json.dumps(build_request(get_corr, OP_GET, get_data), indent=2))
        get_resp = await send(client, args.via, OP_GET, get_data, get_corr)
```

```
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | homework_demo.py (via kafka) =====
request topic = s7875_notice_requests   response topic = s7875_notice_responses
2026-10-27T11:46:53.445-07:00 [CLIENT] listening for responses on s7875_notice_responses

----- Step 1: create one correlation_id for the CREATE_NOTICE request
correlation_id = 007c1cf0-c86c-4024-a84d-fa942ebc9c5f

----- Step 2: send CREATE_NOTICE to s7875_notice_requests

----- Step 3: the request message (as written to the request topic)
{
  "correlation_id": "007c1cf0-c86c-4024-a84d-fa942ebc9c5f",
  "reply_to": "s7875_notice_responses",
  "operation": "CREATE_NOTICE",
  "data": {
    "product": "Organic Baby Spinach 5oz",
    "notice_code": "RCL-2026-07875",
    "units_affected": 4471,
    "manufacturer_id": 1,
    "email": "recalls@greenfields-foods.com",
    "description": "Routine testing found Listeria monocytogenes in lot L4471. Bags were sold in California and Nevada stores.",
    "category": "bacterialContamination"
  }
}

----- Step 4: worker receives and processes it   (see merged log in step 9)

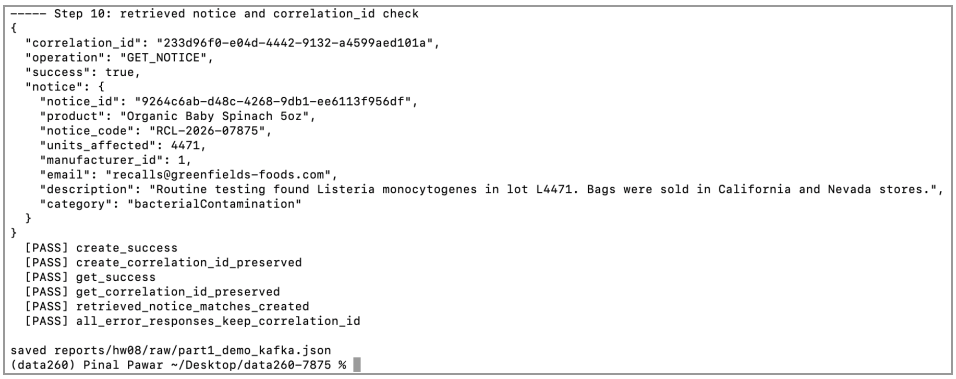
----- Step 5: worker replies on s7875_notice_responses   (see merged log in step 9)
2026-10-27T11:46:53.445-07:00 [CLIENT] SEND           corr=007c1cf0-c86c-4024-a84d-fa942ebc9c5f topic=s7875_notice_requests {"correlation_id": "007c1cf0-c86c-4024-a84d-fa942ebc9c5f", "reply_to": "s7875_notic
e_responses", "operation": "CREATE_NOTICE", "data": {"product": "Organic Baby Spinach 5oz", "notice_code": "RCL-2026-07875", "units_affected": 4471, "manufacturer_id": 1, "email": "recalls@greenfields-foods.c
om", "description": "Routine testing found Listeria monocytogenes in lot L4471. Bags were sold in California and Nevada stores.", "category": "bacterialContamination"}}
2026-10-27T11:46:53.478-07:00 [CLIENT] CLIENT-RESPONSE corr=007c1cf0-c86c-4024-a84d-fa942ebc9c5f topic=s7875_notice_responses p=0 off=6 {"correlation_id": "007c1cf0-c86c-4024-a84d-fa942ebc9c5f", "operation":
"CREATE_NOTICE", "success": true, "notice_id": "9264c6ab-d48c-4268-9db1-ee6113f956df", "message": "Notice created"}

----- Step 6: response received by the client
{
  "correlation_id": "007c1cf0-c86c-4024-a84d-fa942ebc9c5f",
  "operation": "CREATE_NOTICE",
  "success": true,
  "notice_id": "9264c6ab-d48c-4268-9db1-ee6113f956df",
  "message": "Notice created"
}

----- Step 7: extract the generated notice_id
notice_id = 9264c6ab-d48c-4268-9db1-ee6113f956df

----- Step 8: send GET_NOTICE for that notice_id (new correlation_id for this request)
correlation_id = 233d96f0-e04d-4442-9132-a4599aed101a
{
  "correlation_id": "233d96f0-e04d-4442-9132-a4599aed101a",
  "reply_to": "s7875_notice_responses",
  "operation": "GET_NOTICE",
  "data": {
    "notice_id": "9264c6ab-d48c-4268-9db1-ee6113f956df"
  }
}
2026-10-27T11:46:53.478-07:00 [CLIENT] SEND           corr=233d96f0-e04d-4442-9132-a4599aed101a topic=s7875_notice_requests {"correlation_id": "233d96f0-e04d-4442-9132-a4599aed101a", "reply_to": "s7875_notic
e_responses", "operation": "GET_NOTICE", "data": {"notice_id": "9264c6ab-d48c-4268-9db1-ee6113f956df"}}
2026-10-27T11:46:53.485-07:00 [CLIENT] CLIENT-RESPONSE corr=233d96f0-e04d-4442-9132-a4599aed101a topic=s7875_notice_responses p=0 off=7 {"correlation_id": "233d96f0-e04d-4442-9132-a4599aed101a", "operation":
"GET_NOTICE", "success": true, "notice": {"notice_id": "9264c6ab-d48c-4268-9db1-ee6113f956df", "product": "Organic Baby Spinach 5oz", "notice_code": "RCL-2026-07875", "units_affected": 4471, "manufacturer_id"
: 1, "email": "recalls@greenfields-foods.com", "description": "Routine testing found Listeria monocytogenes in lot L4471. Bags were sold in California and Nevada stores.", "category": "bacterialContamination"
}}
```

python homework_demo.py (Kafka client): steps 1-8 - request on s7875_notice_requests, CREATE response with generated notice_id, GET request.



Step 10: retrieved notice and the six checks (both correlation_ids preserved, notice equals the one created, error responses keep their ids).

1.5 Kafka message flow and full sequence CREATE → GET

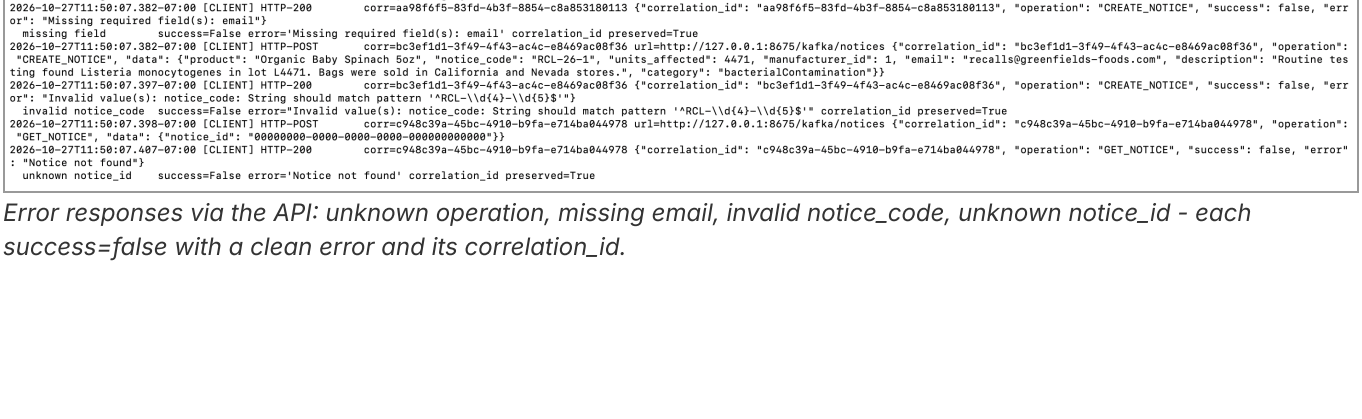
Step 9 merges the client's lines with the worker's (and API's) container logs by timestamp. Kafka-client run, 11:46:53 (worker times are UTC, +7 h):

RUN_LOG.txt excerpt (corr= column and long JSON shortened)

```
### CREATE_NOTICE  correlation_id=007c1cf0-c86c-4024-a84d-fa942ebc9c5f
11:46:53.445-07:00 [CLIENT] SEND            topic=s7875_notice_requests {..."operation": "CREATE_NOTIC
E"...}
18:46:53.468+00:00 [WORKER] RECEIVE         topic=s7875_notice_requests p=0 off=6
18:46:53.470+00:00 [WORKER] PROCESS         operation=CREATE_NOTICE success=True notices_in_store=2
18:46:53.470+00:00 [WORKER] REPLY           topic=s7875_notice_responses {..."notice_id": "9264c6ab
-..."}
11:46:53.478-07:00 [CLIENT] CLIENT-RESPONSE topic=s7875_notice_responses p=0 off=6 {"success": true,
...}
### GET_NOTICE  correlation_id=233d96f0-e04d-4442-9132-a4599aed101a
11:46:53.478-07:00 [CLIENT] SEND            topic=s7875_notice_requests {..."notice_id": "9264c6ab
-..."}
18:46:53.481+00:00 [WORKER] RECEIVE         topic=s7875_notice_requests p=0 off=7
18:46:53.482+00:00 [WORKER] PROCESS         operation=GET_NOTICE success=True notices_in_store=2
18:46:53.482+00:00 [WORKER] REPLY           topic=s7875_notice_responses {"notice": {...}}
11:46:53.485-07:00 [CLIENT] CLIENT-RESPONSE topic=s7875_notice_responses p=0 off=7 {"success": true,
...}
```



python homework_demo.py --via api: the same sequence through the FastAPI wrapper - HTTP-POST → API HTTP-REQUEST → API SEND → WORKER RECEIVE / PROCESS / REPLY → API CLIENT-RESPONSE → HTTP-RESPONSE → client HTTP-200, one correlation_id per request throughout.



Error responses via the API: unknown operation, missing email, invalid notice_code, unknown notice_id - each success=false with a clean error and its correlation_id.

Run	Operation	Sequence (first line → last)	Result
Kafka client	CREATE_NOTICE	5 lines, 33 ms	success, notice_id 9264c6ab-...
Kafka client	GET_NOTICE	5 lines, 7 ms	success, same notice
via FastAPI	CREATE_NOTICE	9 lines, 32 ms	success, notice_id c40cb0b5-...
via FastAPI	GET_NOTICE	9 lines, 28 ms	success, same notice

Part 2 - Three-Agent Kafka Pipeline

2.1 Topics and their creation (Kafka CLI)

Sender	Reads topic	Sends to
You (send_question.py)	-	s7875_inbox (a question)
Planner (planner.py)	s7875_inbox	s7875_tasks (a plan) - 2 partitions
Writer (writer.py)	s7875_tasks	s7875_drafts (a draft answer)
Reviewer (reviewer.py)	s7875_drafts	s7875_final (status + answer)

scripts/hw08_create_topics.sh (lines 10-22)

```
create() {  # name partitions
  echo "+ kafka-topics --create --topic $1 --partitions $2 --replication-factor 1"
  # kafka-topics warns about '_' in names (metric-name collision with '.'); hide that line
  $KT --create --if-not-exists --topic "$1" --partitions "$2" --replication-factor 1 2>&1 \
    | grep -v "WARNING: Due to limitations in metric names" || true
}
create s7875_notice_requests 1
create s7875_notice_responses 1
create s7875_inbox 1
create s7875_tasks 2
create s7875_drafts 1
create s7875_final 1
create s7875_dead_letter 1
```

```
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | creating HW8 topics =====
+ kafka-topics --create --topic s7875_notice_requests --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_notice_responses --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_inbox --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_tasks --partitions 2 --replication-factor 1
+ kafka-topics --create --topic s7875_drafts --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_final --partitions 1 --replication-factor 1
+ kafka-topics --create --topic s7875_dead_letter --partitions 1 --replication-factor 1

+ kafka-topics --list
s7875_dead_letter
s7875_drafts
s7875_final
s7875_inbox
s7875_notice_requests
s7875_notice_responses
s7875_tasks

+ kafka-topics --describe (s7875_*)
Topic: s7875_notice_requests    TopicId: qlsscQUMTYOzBIL-sr8cRA PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s7875_notice_responses   TopicId: zzQIBOsGTEiPUIFkxLPY-g PartitionCount: 1       ReplicationFactor: 1    Configs:
Topic: s7875_inbox      TopicId: SPr8QbexTB6SBeFapXO80g PartitionCount: 1      ReplicationFactor: 1    Configs:
Topic: s7875_tasks      TopicId: 4QnW2KE5R92XpLHeqiJHDQ PartitionCount: 2      ReplicationFactor: 1    Configs:
Topic: s7875_drafts     TopicId: LaKKn5zYQBuzmjejZ4AHxw PartitionCount: 1      ReplicationFactor: 1    Configs:
Topic: s7875_final      TopicId: 8TDDMc5oSY65zIMTgZaFow PartitionCount: 1      ReplicationFactor: 1    Configs:
Topic: s7875_dead_letter        TopicId: cnrARzaHTH2FFONtV8iWcQ PartitionCount: 1       ReplicationFactor: 1    Configs:
(data260) Pinal Pawar ~/Desktop/data260-7875 %
```

bash scripts/hw08_create_topics.sh: kafka-topics --create for each topic, --list, --describe (s7875_tasks PartitionCount 2).

2.2 Agent code

Each agent is its own process (`python planner.py` etc.) running the shared consumer loop in `kafka_hw8/agent_base.py`; the only thing an agent does with another agent is write to a topic. Every message keeps the question's correlation_id and each agent appends one `trace` entry (topic, partition, offset, time).

kafka_hw8/agents.py (lines 103-125)

```python
class Planner(KafkaAgent):
    name = "planner"
    in_topic = INBOX_TOPIC
    group = PLANNER_GROUP

    async def handle(self, msg, record):
        question = msg.get("question") if isinstance(msg, dict) else None
        if not isinstance(question, str) or not question.strip():
            raise ValueError("question is missing or empty")
        reply = await asyncio.to_thread(chat, PLANNER_SYSTEM, question)
        plan = parse_plan(reply)
        if not plan:
            raise ValueError("model returned an empty plan")
        out = {
            "correlation_id": msg["correlation_id"],
            "stage": "plan",
            "question": question,
            "plan": plan,
            "trace": msg.get("trace", []) + [trace_entry(self, record)],
        }
        return [Out(TASKS_TOPIC, msg["correlation_id"], out)]
```

kafka_hw8/agents.py (lines 128-152)

```python
class Writer(KafkaAgent):
    name = "writer"
    in_topic = TASKS_TOPIC
    group = WRITER_GROUP

    async def handle(self, msg, record):
        plan = msg.get("plan") if isinstance(msg, dict) else None
        question = msg.get("question") if isinstance(msg, dict) else None
        if not isinstance(plan, list) or not plan or not all(isinstance(p, str) for p in plan):
            raise ValueError("task has no usable plan (expected a non-empty list of steps)")
        if not isinstance(question, str) or not question.strip():
            raise ValueError("task has no question")
        prompt = (f"Question: {question}\n\nPlan:\n" + "\n".join(f"- {p}" for p in plan)
                  + "\n\nWrite the answer now.")
        draft = (await asyncio.to_thread(chat, WRITER_SYSTEM, prompt)).strip()
        if not draft:
            raise ValueError("model returned an empty draft")
        out = {**{k: msg[k] for k in ("correlation_id", "question", "plan")},
               "stage": "draft", "draft": draft,
               "trace": msg.get("trace", []) + [trace_entry(self, record)]}
        return [Out(DRAFTS_TOPIC, msg["correlation_id"], out)]
```

kafka_hw8/agents.py (lines 169-203)

```python
class Reviewer(KafkaAgent):
    name = "reviewer"
    in_topic = DRAFTS_TOPIC
    group = REVIEWER_GROUP

    async def handle(self, msg, record):
        draft = msg.get("draft") if isinstance(msg, dict) else None
        if not isinstance(draft, str) or not draft.strip():
            raise ValueError("draft is missing or empty")
        words = len(draft.split())
        unsafe = unsafe_phrase(draft)
        if words > MAX_DRAFT_WORDS:  # rule checks first; no model call needed
            status, reason = "rejected", f"draft too long ({words} words > {MAX_DRAFT_WORDS})"
        elif unsafe:
            status, reason = "rejected", (f"rule check: draft calls a recalled product safe "
                                          f"(\"{unsafe}\")")
        else:
            prompt = (f"Question: {msg.get('question')}\n\nPlan:\n"
                      + "\n".join(f"- {p}" for p in msg.get("plan", []))
                      + f"\n\nDraft answer:\n{draft}")
            status, reason = parse_review(await asyncio.to_thread(chat, REVIEWER_SYSTEM, prompt))
        final = {
            "correlation_id": msg["correlation_id"],
            "status": status,
            "answer": draft,
            "review": reason,
            "question": msg.get("question"),
            "plan": msg.get("plan"),
            "model": model_label(),
            "trace": msg.get("trace", []) + [trace_entry(self, record)],
            "finished_at": now(),
        }
        return [Out(FINAL_TOPIC, msg["correlation_id"], final)]
```

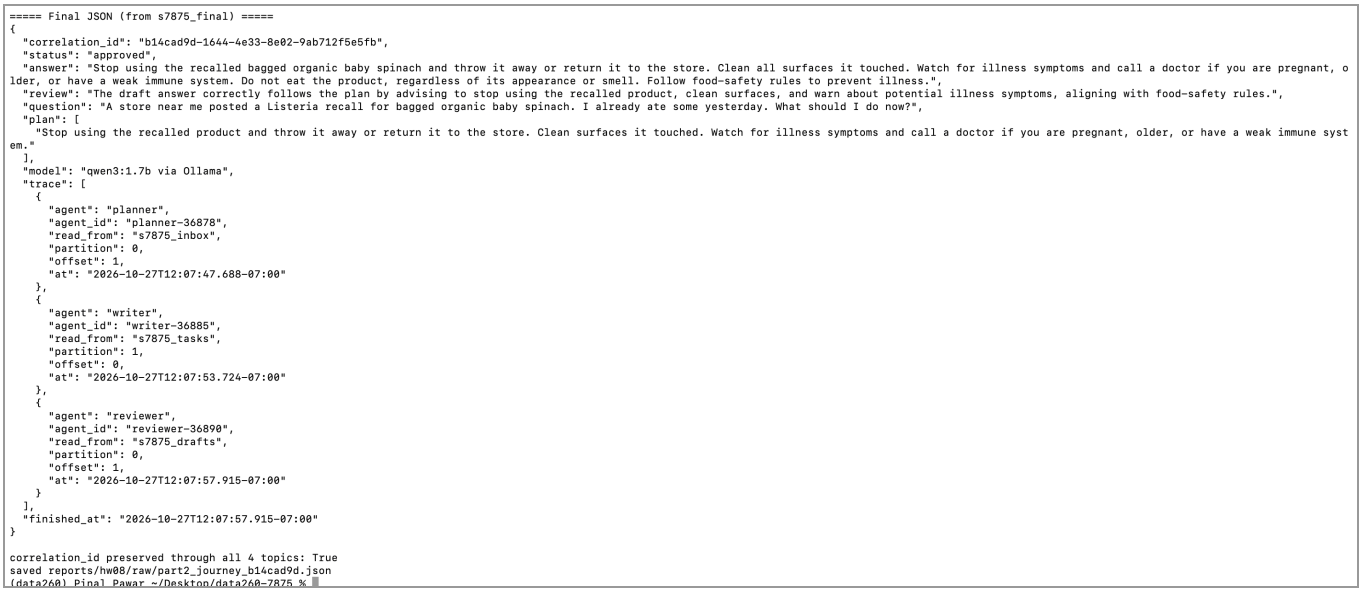
2.3 One complete journey s7875_inbox → s7875_final (run 2, qwen3:1.7b)

raw/part2_run2_planner|writer|reviewer.log (one line per stage; full logs in raw/)

```
PLANNER  12:07:41.749 RECEIVE  topic=s7875_inbox  p=0 off=1   corr=b14cad9d-1644-4e33-8e02-9ab712f5e5f
b
PLANNER  12:07:47.687 PROCESS-DONE steps=1 model=qwen3:1.7b via Ollama
PLANNER  12:07:47.688 SEND     topic=s7875_tasks              -> COMMIT p=0 next_offset=2
WRITER   12:07:47.857 RECEIVE  topic=s7875_tasks  p=1 off=0   corr=b14cad9d-...
WRITER   12:07:53.724 PROCESS-DONE words=59
WRITER   12:07:53.724 SEND     topic=s7875_drafts             -> COMMIT p=1 next_offset=1
REVIEWER 12:07:53.806 RECEIVE  topic=s7875_drafts p=0 off=1   corr=b14cad9d-...
REVIEWER 12:07:57.915 PROCESS-DONE status=approved
REVIEWER 12:07:57.915 SEND     topic=s7875_final              -> COMMIT p=0 next_offset=2
SENDER   12:07:57.957 FINAL    topic=s7875_final  p=0 off=1   corr=b14cad9d-...   (16.2 s end to end)
```



send_question.py: the message on all four topics with the same correlation_id, and the final message read from s7875_final.



Final JSON: status "approved", answer, review, plan, model, and the trace planner (inbox p0/1) → writer (tasks p1/0) → reviewer (drafts p0/1).

Run 1 (11:57, `raw/part2_journey_d5f6857e.json`) went through the same four topics correctly, but the model's answer said recalled spinach "may still be safe" and the Reviewer approved it. Safety rules and a Reviewer rule check were added before run 2 (see AI use).

2.4 How Kafka helped the agents coordinate

- **No agent knows another exists.** Each one only knows its input and output topic, so the Writer could be stopped, restarted or run twice without the Planner or Reviewer changing at all (Part 3 does exactly this).
- **Work waits instead of failing.** A question sent while the Writer is down stays in s7875_tasks at its offset and is processed when the Writer comes back; in the HW2 graph a missing step meant a failed call.
- **Scaling by partitions.** s7875_tasks has two partitions, so two Writers in one consumer group share the load automatically (Part 3 ordering run); Kafka rebalanced the partitions when the second Writer joined.
- **One id ties it together.** The correlation_id is also the message key and is copied into every message, so a whole journey can be followed across processes and topics, and the final message carries its own trace.

Part 3 - Failure Behavior

Every fix lives in the shared loop `kafka_hw8/agent_base.py` behind a switch, so the same code shows the before and after behaviour: `IDEMPOTENCY`, `COMMIT_MODE`, `MAX_RETRIES`, and the producer's partition key. `python scripts/run_hw08_part3.py duplicate|kill|poison|ordering` starts real Writer processes against Kafka, each phase in a fresh consumer group whose offsets start "now". The Writer uses an offline model stand-in here, since these tests measure Kafka delivery, not model output.

3.1 Failure table (real runs, 27 Oct 2026)

Failure scenario	Before fix	After fix
Duplicate delivery (same task replayed, same correlation_id)	IDEMPOTENCY=0: 2 receives → **2 model calls, 2 drafts** (extra side effect)	IDEMPOTENCY=1: 2 receives → **1 model call, 1 draft**, second copy DUPLICATE-SKIP and committed
Consumer killed mid-work (kill -9 during an 8 s task)	COMMIT_MODE=before: offset committed on receipt (6 > message offset 5); restarted Writer never saw it → **message lost**, 0 drafts	COMMIT_MODE=after: committed offset still 1 = message offset; restarted Writer got it again → **completed**, 1 draft, committed 2
Poison message (plan: null, a good task behind it on p0)	MAX_RETRIES=0: Writer **crashed 3 times on offset 6**; committed offset stuck at 6; good message **never processed**	MAX_RETRIES=3: 3 attempts (0.5 s, 1 s backoff) → 1 record on **s7875_dead_letter**; offset committed; good message **processed**
Two partitions / two consumers (20 messages, VERIFY_SEED 267875)	no key (round-robin p0/p1): **2 of 4 notices out of order** (59801: seq 2 before 1; 61335: seq 4 before 3)	key = notice_code: **0 of 4 out of order**; each notice on one partition and one Writer

3.2 Duplicate delivery - idempotency

kafka_hw8/agent_base.py (lines 177-181) - skip a key already processed

```
        if s.idempotency and self.store.seen(key):
            log(self.tag, "DUPLICATE-SKIP", corr, key=key,
                note="already processed; no model call, nothing sent")
            await self._commit_after(tp, record, corr)
            return
```

kafka_hw8/agent_base.py (lines 71-79) - keys kept in SQLite so they survive a restart

```
class ProcessedStore:

    def __init__(self, path: Path):
        path.parent.mkdir(parents=True, exist_ok=True)
        self.db = sqlite3.connect(path)
        self.db.execute("CREATE TABLE IF NOT EXISTS processed (key TEXT PRIMARY KEY, "
                        "topic TEXT, partition INTEGER, offset INTEGER, at TEXT, outcome TEXT)")
        self.db.commit()
```

```
===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | run_hw08_part3.py duplicate (run 122204) =====

================================================================================================
2026-10-27T12:22:04.983-07:00  DUPLICATE DELIVERY — before fix (IDEMPOTENCY=0)
================================================================================================
2026-10-27T12:22:05.241-07:00 [PART3] started writer-A (pid 37535) group=s7875_writers_p3_duplicate_before_122204 IDEMPOTENCY=0  log=part3_duplicate_before_writer-A.log
    | ===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | writer (writer-A) =====
    | 2026-10-27T12:22:05.370-07:00 [WRITER-A] in=s7875_tasks group=s7875_writers_p3_duplicate_before_122204 idempotency=False commit_mode=after max_retries=3 work_delay_s=0.0 dead_letter=s7875_de
    | 2026-10-27T12:22:05.398-07:00 [WRITER-A] ASSIGNED partitions [0, 1]
    | 2026-10-27T12:22:05.401-07:00 [WRITER-A] writer is ready and listening on s7875_tasks
2026-10-27T12:22:05.447-07:00 [PART3] send task corr=dup-before-8c578dd3, then REPLAY the identical message (same correlation_id)
    | 2026-10-27T12:22:05.528-07:00 [WRITER-A] RECEIVE         corr=dup-before-8c578dd3 topic=s7875_tasks p=1 off=26 key=dup-before-8c578dd3 {"correlation_id": "dup-before-8c578dd3", "stage": "pla
    | 2026-10-27T12:22:05.528-07:00 [WRITER-A] PROCESS-START   corr=dup-before-8c578dd3 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:22:05.528-07:00 [WRITER-A] PROCESS-DONE    corr=dup-before-8c578dd3 words=51 model=offline stand-in (HW8_FAKE_LLM=1)
    | 2026-10-27T12:22:05.528-07:00 [WRITER-A] SEND            corr=dup-before-8c578dd3 topic=s7875_drafts {"correlation_id": "dup-before-8c578dd3", "question": "What should I do with a recalled b
2026-10-27T12:22:05.533-07:00 [PART3] both copies written at (partition, offset) = [(1, 26), (1, 27)]
    | 2026-10-27T12:22:05.551-07:00 [WRITER-A] COMMIT          corr=dup-before-8c578dd3 p=1 next_offset=27
    | 2026-10-27T12:22:05.553-07:00 [WRITER-A] RECEIVE         corr=dup-before-8c578dd3 topic=s7875_tasks p=1 off=27 key=dup-before-8c578dd3 {"correlation_id": "dup-before-8c578dd3", "stage": "pla
    | 2026-10-27T12:22:05.553-07:00 [WRITER-A] PROCESS-START   corr=dup-before-8c578dd3 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:22:05.553-07:00 [WRITER-A] PROCESS-DONE    corr=dup-before-8c578dd3 words=51 model=offline stand-in (HW8_FAKE_LLM=1)
    | 2026-10-27T12:22:05.553-07:00 [WRITER-A] SEND            corr=dup-before-8c578dd3 topic=s7875_drafts {"correlation_id": "dup-before-8c578dd3", "question": "What should I do with a recalled b
    | 2026-10-27T12:22:05.559-07:00 [WRITER-A] COMMIT          corr=dup-before-8c578dd3 p=1 next_offset=28
    | 2026-10-27T12:22:10.876-07:00 [WRITER-A] stopping (signal)
2026-10-27T12:22:10.913-07:00 [PART3] RESULT before: {"group": "s7875_writers_p3_duplicate_before_122204", "correlation_id": "dup-before-8c578dd3", "copies_sent": 2, "placed": [[1, 26], [1, 27]], "received":
2, "model_calls": 2, "duplicate_skips": 0, "drafts_on_s7875_drafts": 2, "log": "part3_duplicate_before_writer-A.log"}

================================================================================================
2026-10-27T12:22:10.913-07:00  DUPLICATE DELIVERY — after fix (IDEMPOTENCY=1)
================================================================================================
2026-10-27T12:22:11.163-07:00 [PART3] started writer-A (pid 37538) group=s7875_writers_p3_duplicate_after_122204 IDEMPOTENCY=1  log=part3_duplicate_after_writer-A.log
    | ===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | writer (writer-A) =====
    | 2026-10-27T12:22:11.283-07:00 [WRITER-A] in=s7875_tasks group=s7875_writers_p3_duplicate_after_122204 idempotency=True commit_mode=after max_retries=3 work_delay_s=0.0 dead_letter=s7875_dead
    | 2026-10-27T12:22:11.308-07:00 [WRITER-A] ASSIGNED partitions [0, 1]
    | 2026-10-27T12:22:11.312-07:00 [WRITER-A] writer is ready and listening on s7875_tasks
2026-10-27T12:22:11.365-07:00 [PART3] send task corr=dup-after-61a4cf25, then REPLAY the identical message (same correlation_id)
    | 2026-10-27T12:22:11.378-07:00 [WRITER-A] RECEIVE         corr=dup-after-61a4cf25 topic=s7875_tasks p=1 off=28 key=dup-after-61a4cf25 {"correlation_id": "dup-after-61a4cf25", "stage": "plan",
    | 2026-10-27T12:22:11.378-07:00 [WRITER-A] PROCESS-START   corr=dup-after-61a4cf25 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:22:11.379-07:00 [WRITER-A] PROCESS-DONE    corr=dup-after-61a4cf25 words=51 model=offline stand-in (HW8_FAKE_LLM=1)
    | 2026-10-27T12:22:11.379-07:00 [WRITER-A] SEND            corr=dup-after-61a4cf25 topic=s7875_drafts {"correlation_id": "dup-after-61a4cf25", "question": "What should I do with a recalled bag
2026-10-27T12:22:11.380-07:00 [PART3] both copies written at (partition, offset) = [(1, 28), (1, 29)]
    | 2026-10-27T12:22:11.387-07:00 [WRITER-A] COMMIT          corr=dup-after-61a4cf25 p=1 next_offset=29
    | 2026-10-27T12:22:11.389-07:00 [WRITER-A] RECEIVE         corr=dup-after-61a4cf25 topic=s7875_tasks p=1 off=29 key=dup-after-61a4cf25 {"correlation_id": "dup-after-61a4cf25", "stage": "plan",
    | 2026-10-27T12:22:11.389-07:00 [WRITER-A] DUPLICATE-SKIP  corr=dup-after-61a4cf25 key=dup-after-61a4cf25 note=already processed; no model call, nothing sent
    | 2026-10-27T12:22:11.390-07:00 [WRITER-A] COMMIT          corr=dup-after-61a4cf25 p=1 next_offset=30
    | 2026-10-27T12:22:16.715-07:00 [WRITER-A] stopping (signal)
2026-10-27T12:22:16.787-07:00 [PART3] RESULT after: {"group": "s7875_writers_p3_duplicate_after_122204", "correlation_id": "dup-after-61a4cf25", "copies_sent": 2, "placed": [[1, 28], [1, 29]], "received": 2,
"model_calls": 1, "duplicate_skips": 1, "drafts_on_s7875_drafts": 1, "log": "part3_duplicate_after_writer-A.log"}

================================================================================================
2026-10-27T12:22:16.791-07:00  FAILURE TABLE
================================================================================================
| Failure scenario | Before fix | After fix |
|---|---|---|
| Duplicate delivery | same task sent twice -> 2 model calls, 2 drafts (extra side effect) | 1 model call, 1 draft, 1 DUPLICATE-SKIP |
| Consumer killed mid-work | offset committed on receipt (committed=6 > msg offset 5); after restart redelivered=False -> LOST (never completed) | commit after send (committed=1 at kill); redelivered=True ->
completed after restart |
| Poison message | Writer crashed 3x on the same offset; good message processed=False (blocked) | 3 attempts -> s7875_dead_letter (1 record); good message processed=True |
| Two partitions/two consumers | no key: 2/4 notices processed out of order (2 inversions) | key=notice_code: 0/4 out of order; each notice on one partition |
(data260) Pinal Pawar ~/Desktop/data260-7875 %
```

run_hw08_part3.py duplicate: before - both copies processed (2 SEND to s7875_drafts); after - second copy DUPLICATE-SKIP, 1 draft. The failure table below it has all four rows.

3.3 Consumer death - commit only after success

kafka_hw8/agent_base.py (lines 172-175) - the 'before' behaviour

```
        if s.commit_mode == "before":
            await self.consumer.commit({tp: record.offset + 1})
            log(self.tag, "COMMIT", corr, p=record.partition, next_offset=record.offset + 1,
                note="committed BEFORE processing (COMMIT_MODE=before)")
```

kafka_hw8/agent_base.py (lines 224-235) - after fix: send outputs, record the key, then commit offset+1

```
        for out in outputs:
            log(self.tag, "SEND", out.value.get("correlation_id"), out.value, topic=out.topic)
            await self.producer.send_and_wait(out.topic, json.dumps(out.value).encode(),
                                              key=out.key.encode() if out.key else None)
        if s.idempotency:
            self.store.mark(key, record, outcome)
        await self._commit_after(tp, record, corr)

    async def _commit_after(self, tp, record, corr) -> None:
        if self.settings.commit_mode == "after":
            await self.consumer.commit({tp: record.offset + 1})
            log(self.tag, "COMMIT", corr, p=record.partition, next_offset=record.offset + 1)
```

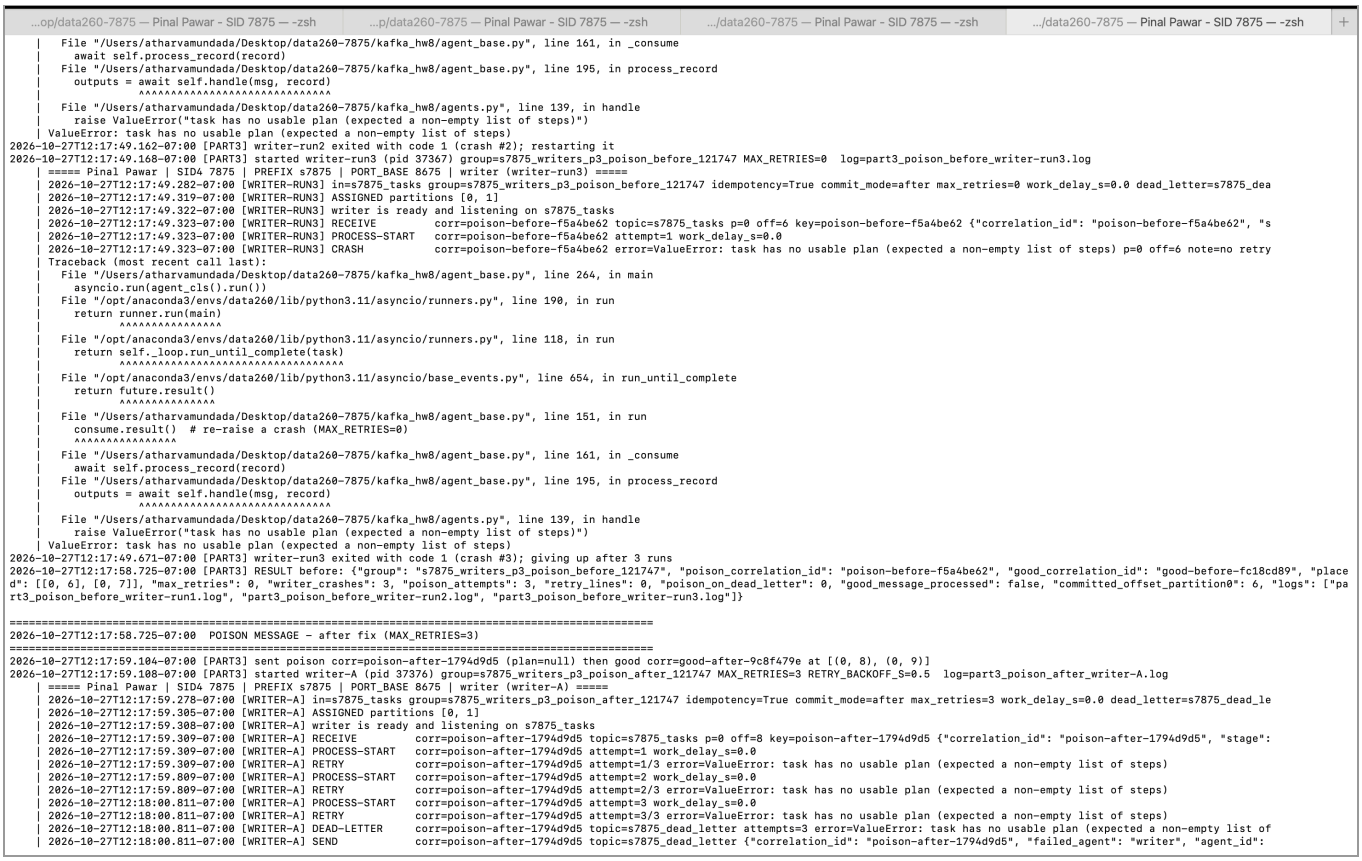


run_hw08_part3.py kill: before - COMMIT then PROCESS-START, kill -9, committed offset 6 > 5, writer-B gets nothing (LOST); after - kill -9 with committed offset still 1, writer-B RECEIVEs offset 1 again, SENDs the draft and commits 2.

3.4 Poison message - bounded retries and s7875_dead_letter

kafka_hw8/agent_base.py (lines 186-222) - retry up to MAX_RETRIES, then dead-letter

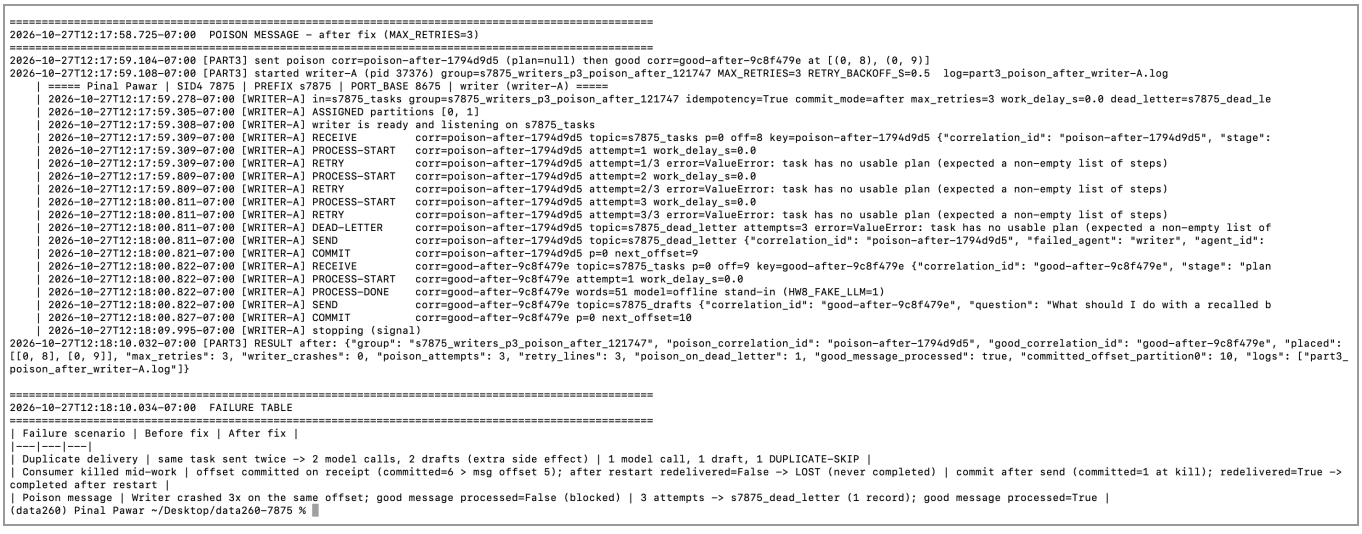
```
        while True:
            attempt += 1
            try:
                if decode_error:
                    raise ValueError(f"message is not valid JSON: {decode_error}")
                outputs = await self.handle(msg, record)
                break
            except Exception as exc:  # noqa: BLE001 - every failure is retried or dead-lettered
                err = f"{type(exc).__name__}: {exc}"
                if s.max_retries <= 0:
                    log(self.tag, "CRASH", corr, error=err, p=record.partition,
                        off=record.offset, note="no retry handling (MAX_RETRIES=0); "
                        "offset not committed")
                    raise
                log(self.tag, "RETRY", corr, attempt=f"{attempt}/{s.max_retries}", error=err)
                if attempt >= s.max_retries:
                    dead = {
                        "correlation_id": corr,
                        "failed_agent": self.name,
                        "agent_id": self.agent_id,
                        "error": err,
                        "attempts": attempt,
                        "source": {"topic": record.topic, "partition": record.partition,
                                   "offset": record.offset},
                        "original": msg if msg is not None else _raw(record.value),
                        "dead_lettered_at": now(),
                    }
                    outputs = [Out(DEAD_LETTER_TOPIC, corr, dead)]
                    outcome = "dead_letter"
                    log(self.tag, "DEAD-LETTER", corr, topic=DEAD_LETTER_TOPIC,
                        attempts=attempt, error=err)
                    break
                await asyncio.sleep(s.retry_backoff_s * attempt)
```



```
...op/data260-7875 — Pinal Pawar - SID 7875 — -zsh  ...p/data260-7875 — Pinal Pawar - SID 7875 — -zsh  .../data260-7875 — Pinal Pawar - SID 7875 — -zsh  .../data260-7875 — Pinal Pawar - SID 7875 — -zsh
    |   File "/Users/atharvamundada/Desktop/data260-7875/kafka_hw8/agent_base.py", line 161, in _consume
    |     await self.process_record(record)
    |   File "/Users/atharvamundada/Desktop/data260-7875/kafka_hw8/agent_base.py", line 195, in process_record
    |     outputs = await self.handle(msg, record)
    |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |   File "/Users/atharvamundada/Desktop/data260-7875/kafka_hw8/agents.py", line 139, in handle
    |     raise ValueError("task has no usable plan (expected a non-empty list of steps)")
    | ValueError: task has no usable plan (expected a non-empty list of steps)
2026-10-27T12:17:49.162-07:00 [PART3] writer-run2 exited with code 1 (crash #2); restarting it
2026-10-27T12:17:49.168-07:00 [PART3] started writer-run3 (pid 37367) group=s7875_writers_p3_poison_before_121747 MAX_RETRIES=0  log=part3_poison_before_writer-run3.log
    | ===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | writer (writer-run3) =====
    | 2026-10-27T12:17:49.282-07:00 [WRITER-RUN3] in=s7875_tasks group=s7875_writers_p3_poison_before_121747 idempotency=True commit_mode=after max_retries=0 work_delay_s=0.0 dead_letter=s7875_dea
    | 2026-10-27T12:17:49.319-07:00 [WRITER-RUN3] ASSIGNED partitions [0, 1]
    | 2026-10-27T12:17:49.322-07:00 [WRITER-RUN3] writer is ready and listening on s7875_tasks
    | 2026-10-27T12:17:49.323-07:00 [WRITER-RUN3] RECEIVE         corr=poison-before-f5a4be62 topic=s7875_tasks p=0 off=6 key=poison-before-f5a4be62 {"correlation_id": "poison-before-f5a4be62", "s
    | 2026-10-27T12:17:49.323-07:00 [WRITER-RUN3] PROCESS-START   corr=poison-before-f5a4be62 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:17:49.323-07:00 [WRITER-RUN3] CRASH           corr=poison-before-f5a4be62 error=ValueError: task has no usable plan (expected a non-empty list of steps) p=0 off=6 note=no retry
    | Traceback (most recent call last):
    |   File "/Users/atharvamundada/Desktop/data260-7875/kafka_hw8/agent_base.py", line 264, in main
    |     asyncio.run(agent_cls().run())
    |   File "/opt/anaconda3/envs/data260/lib/python3.11/asyncio/runners.py", line 190, in run
    |     return runner.run(main)
    |            ^^^^^^^^^^^^^^^^
    |   File "/opt/anaconda3/envs/data260/lib/python3.11/asyncio/runners.py", line 118, in run
    |     return self._loop.run_until_complete(task)
    |            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |   File "/opt/anaconda3/envs/data260/lib/python3.11/asyncio/base_events.py", line 654, in run_until_complete
    |     return future.result()
    |            ^^^^^^^^^^^^^^^
    |   File "/Users/atharvamundada/Desktop/data260-7875/kafka_hw8/agent_base.py", line 151, in run
    |     consume.result()  # re-raise a crash (MAX_RETRIES=0)
    |     ^^^^^^^^^^^^^^^^
    |   File "/Users/atharvamundada/Desktop/data260-7875/kafka_hw8/agent_base.py", line 161, in _consume
    |     await self.process_record(record)
    |   File "/Users/atharvamundada/Desktop/data260-7875/kafka_hw8/agent_base.py", line 195, in process_record
    |     outputs = await self.handle(msg, record)
    |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |   File "/Users/atharvamundada/Desktop/data260-7875/kafka_hw8/agents.py", line 139, in handle
    |     raise ValueError("task has no usable plan (expected a non-empty list of steps)")
    | ValueError: task has no usable plan (expected a non-empty list of steps)
2026-10-27T12:17:49.671-07:00 [PART3] writer-run3 exited with code 1 (crash #3); giving up after 3 runs
2026-10-27T12:17:58.725-07:00 [PART3] RESULT before: {"group": "s7875_writers_p3_poison_before_121747", "poison_correlation_id": "poison-before-f5a4be62", "good_correlation_id": "good-before-fc18cd89", "place
d": [[0, 6], [0, 7]], "max_retries": 0, "writer_crashes": 3, "poison_attempts": 3, "retry_lines": 0, "poison_on_dead_letter": 0, "good_message_processed": false, "committed_offset_partition0": 6, "logs": ["pa
rt3_poison_before_writer-run1.log", "part3_poison_before_writer-run2.log", "part3_poison_before_writer-run3.log"]}

================================================================================
2026-10-27T12:17:58.725-07:00  POISON MESSAGE — after fix (MAX_RETRIES=3)
================================================================================
2026-10-27T12:17:59.104-07:00 [PART3] sent poison corr=poison-after-1794d9d5 (plan=null) then good corr=good-after-9c8f479e at [(0, 8), (0, 9)]
2026-10-27T12:17:59.108-07:00 [PART3] started writer-A (pid 37376) group=s7875_writers_p3_poison_after_121747 MAX_RETRIES=3 RETRY_BACKOFF_S=0.5  log=part3_poison_after_writer-A.log
    | ===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | writer (writer-A) =====
    | 2026-10-27T12:17:59.278-07:00 [WRITER-A] in=s7875_tasks group=s7875_writers_p3_poison_after_121747 idempotency=True commit_mode=after max_retries=3 work_delay_s=0.0 dead_letter=s7875_dead_le
    | 2026-10-27T12:17:59.305-07:00 [WRITER-A] ASSIGNED partitions [0, 1]
    | 2026-10-27T12:17:59.308-07:00 [WRITER-A] writer is ready and listening on s7875_tasks
    | 2026-10-27T12:17:59.309-07:00 [WRITER-A] RECEIVE         corr=poison-after-1794d9d5 topic=s7875_tasks p=0 off=8 key=poison-after-1794d9d5 {"correlation_id": "poison-after-1794d9d5", "stage":
    | 2026-10-27T12:17:59.309-07:00 [WRITER-A] PROCESS-START   corr=poison-after-1794d9d5 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:17:59.309-07:00 [WRITER-A] RETRY           corr=poison-after-1794d9d5 attempt=1/3 error=ValueError: task has no usable plan (expected a non-empty list of steps)
    | 2026-10-27T12:17:59.809-07:00 [WRITER-A] PROCESS-START   corr=poison-after-1794d9d5 attempt=2 work_delay_s=0.0
    | 2026-10-27T12:17:59.809-07:00 [WRITER-A] RETRY           corr=poison-after-1794d9d5 attempt=2/3 error=ValueError: task has no usable plan (expected a non-empty list of steps)
    | 2026-10-27T12:18:00.811-07:00 [WRITER-A] PROCESS-START   corr=poison-after-1794d9d5 attempt=3 work_delay_s=0.0
    | 2026-10-27T12:18:00.811-07:00 [WRITER-A] RETRY           corr=poison-after-1794d9d5 attempt=3/3 error=ValueError: task has no usable plan (expected a non-empty list of steps)
    | 2026-10-27T12:18:00.811-07:00 [WRITER-A] DEAD-LETTER     corr=poison-after-1794d9d5 topic=s7875_dead_letter attempts=3 error=ValueError: task has no usable plan (expected a non-empty list of
    | 2026-10-27T12:18:00.811-07:00 [WRITER-A] SEND            corr=poison-after-1794d9d5 topic=s7875_dead_letter {"correlation_id": "poison-after-1794d9d5", "failed_agent": "writer", "agent_id":
```

Before (MAX_RETRIES=0): the Writer crashes on offset 6 every time it is restarted (crash #2, crash #3, then giving up); RESULT before: 3 crashes, good message processed = false, committed offset stuck at 6. The after-fix run starts below.



```
================================================================================
2026-10-27T12:17:58.725-07:00  POISON MESSAGE — after fix (MAX_RETRIES=3)
================================================================================
2026-10-27T12:17:59.104-07:00 [PART3] sent poison corr=poison-after-1794d9d5 (plan=null) then good corr=good-after-9c8f479e at [(0, 8), (0, 9)]
2026-10-27T12:17:59.108-07:00 [PART3] started writer-A (pid 37376) group=s7875_writers_p3_poison_after_121747 MAX_RETRIES=3 RETRY_BACKOFF_S=0.5  log=part3_poison_after_writer-A.log
    | ===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | writer (writer-A) =====
    | 2026-10-27T12:17:59.278-07:00 [WRITER-A] in=s7875_tasks group=s7875_writers_p3_poison_after_121747 idempotency=True commit_mode=after max_retries=3 work_delay_s=0.0 dead_letter=s7875_dead_le
    | 2026-10-27T12:17:59.305-07:00 [WRITER-A] ASSIGNED partitions [0, 1]
    | 2026-10-27T12:17:59.308-07:00 [WRITER-A] writer is ready and listening on s7875_tasks
    | 2026-10-27T12:17:59.309-07:00 [WRITER-A] RECEIVE         corr=poison-after-1794d9d5 topic=s7875_tasks p=0 off=8 key=poison-after-1794d9d5 {"correlation_id": "poison-after-1794d9d5", "stage":
    | 2026-10-27T12:17:59.309-07:00 [WRITER-A] PROCESS-START   corr=poison-after-1794d9d5 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:17:59.309-07:00 [WRITER-A] RETRY           corr=poison-after-1794d9d5 attempt=1/3 error=ValueError: task has no usable plan (expected a non-empty list of steps)
    | 2026-10-27T12:17:59.809-07:00 [WRITER-A] PROCESS-START   corr=poison-after-1794d9d5 attempt=2 work_delay_s=0.0
    | 2026-10-27T12:17:59.809-07:00 [WRITER-A] RETRY           corr=poison-after-1794d9d5 attempt=2/3 error=ValueError: task has no usable plan (expected a non-empty list of steps)
    | 2026-10-27T12:18:00.811-07:00 [WRITER-A] PROCESS-START   corr=poison-after-1794d9d5 attempt=3 work_delay_s=0.0
    | 2026-10-27T12:18:00.811-07:00 [WRITER-A] RETRY           corr=poison-after-1794d9d5 attempt=3/3 error=ValueError: task has no usable plan (expected a non-empty list of steps)
    | 2026-10-27T12:18:00.811-07:00 [WRITER-A] DEAD-LETTER     corr=poison-after-1794d9d5 topic=s7875_dead_letter attempts=3 error=ValueError: task has no usable plan (expected a non-empty list of
    | 2026-10-27T12:18:00.811-07:00 [WRITER-A] SEND            corr=poison-after-1794d9d5 topic=s7875_dead_letter {"correlation_id": "poison-after-1794d9d5", "failed_agent": "writer", "agent_id":
    | 2026-10-27T12:18:00.821-07:00 [WRITER-A] COMMIT          corr=poison-after-1794d9d5 p=0 next_offset=9
    | 2026-10-27T12:18:00.822-07:00 [WRITER-A] RECEIVE         corr=good-after-9c8f479e topic=s7875_tasks p=0 off=9 key=good-after-9c8f479e {"correlation_id": "good-after-9c8f479e", "stage": "plan
    | 2026-10-27T12:18:00.822-07:00 [WRITER-A] PROCESS-START   corr=good-after-9c8f479e attempt=1 work_delay_s=0.0
    | 2026-10-27T12:18:00.822-07:00 [WRITER-A] PROCESS-DONE    corr=good-after-9c8f479e words=51 model=offline stand-in (HW8_FAKE_LLM=1)
    | 2026-10-27T12:18:00.822-07:00 [WRITER-A] SEND            corr=good-after-9c8f479e topic=s7875_drafts {"correlation_id": "good-after-9c8f479e", "question": "What should I do with a recalled b
    | 2026-10-27T12:18:00.827-07:00 [WRITER-A] COMMIT          corr=good-after-9c8f479e p=0 next_offset=10
    | 2026-10-27T12:18:09.995-07:00 [WRITER-A] stopping (signal)
2026-10-27T12:18:10.032-07:00 [PART3] RESULT after: {"group": "s7875_writers_p3_poison_after_121747", "poison_correlation_id": "poison-after-1794d9d5", "good_correlation_id": "good-after-9c8f479e", "placed":
[[0, 8], [0, 9]], "max_retries": 3, "writer_crashes": 0, "poison_attempts": 3, "retry_lines": 3, "poison_on_dead_letter": 1, "good_message_processed": true, "committed_offset_partition0": 10, "logs": ["part3_
poison_after_writer-A.log"]}

================================================================================
2026-10-27T12:18:10.034-07:00  FAILURE TABLE
================================================================================
| Failure scenario | Before fix | After fix |
|---|---|---|
| Duplicate delivery | same task sent twice -> 2 model calls, 2 drafts (extra side effect) | 1 model call, 1 draft, 1 DUPLICATE-SKIP |
| Consumer killed mid-work | offset committed on receipt (committed=6 > msg offset 5); after restart redelivered=False -> LOST (never completed) | commit after send (committed=1 at kill); redelivered=True ->
completed after restart |
| Poison message | Writer crashed 3x on the same offset; good message processed=False (blocked) | 3 attempts -> s7875_dead_letter (1 record); good message processed=True |
(data260) Pinal Pawar ~/Desktop/data260-7875 %
```

After (MAX_RETRIES=3): RETRY 1/3, 2/3, 3/3 → DEAD-LETTER to s7875_dead_letter → COMMIT 9 → the good message behind it is RECEIVEd, processed and committed (10).

Record on s7875_dead_letter (raw/part3_poison.json)


```
{"correlation_id": "poison-after-1794d9d5", "failed_agent": "writer", "agent_id": "writer-A",
 "error": "ValueError: task has no usable plan (expected a non-empty list of steps)",
 "attempts": 3, "source": {"topic": "s7875_tasks", "partition": 0, "offset": 8},
 "original": {"correlation_id": "poison-after-1794d9d5", "stage": "plan", "plan": null, ...},
 "dead_lettered_at": "2026-10-27T12:18:00.811-07:00"}
```

3.5 Ordering - two partitions, two Writers, then a partition key

20 related messages from `random.Random(267875)`: status updates for four notices, each notice's updates numbered seq 1..n. Two Writers in one consumer group (writer-1 → p0, writer-2 → p1 after the rebalance). Run 1 sends without a key (spread round-robin, i % 2); run 2 keys every message by its notice_code.

scripts/run_hw08_part3.py (lines 378-382) - the only difference between the two runs

```
        if run == "unkeyed":
            placed = await send_json(BOOTSTRAP, TASKS_TOPIC, [(None, m) for m in msgs],
                                     partition_for=lambda i: i % 2)
        else:
            placed = await send_json(BOOTSTRAP, TASKS_TOPIC, [(m["entity"], m) for m in msgs])
```

kafka_hw8/kafka_util.py (lines 134-139) - producer: key -> same partition

```
        for i, (key, value) in enumerate(messages):
            kwargs = {}
            if partition_for is not None:
                kwargs["partition"] = partition_for(i)
            meta = await producer.send_and_wait(
                topic, json.dumps(value).encode(), key=key.encode() if key else None, **kwargs)
```

Notice	Updates	No key: processed seq order	Partitions	Key = notice_code	Partition
RCL-2026-59801	8	**2, 1**, 3, 4, 5, 6, 7, 8 - out of order	0, 1	1, 2, 3, 4, 5, 6, 7, 8	1
RCL-2026-66434	5	1, 2, 3, 4, 5 (in order by luck)	0, 1	1, 2, 3, 4, 5	1
RCL-2026-74265	1	1	0	1	1
RCL-2026-61335	6	1, 2, **4, 3**, 5, 6 - out of order	0, 1	1, 2, 3, 4, 5, 6	0

```
...op/data260-7875 — Pinal Pawar - SID 7875 — -zsh  ...p/data260-7875 — Pinal Pawar - SID 7875 — -zsh  .../data260-7875 — Pinal Pawar - SID 7875 — -zsh  .../data260-7875 — Pinal Pawar - SID 7875 — -zsh
    | 2026-10-27T12:19:20.164-07:00 [WRITER-1] COMMIT          corr=ord-unkeyed-121913-18 p=0 next_offset=20
    | 2026-10-27T12:19:20.269-07:00 [WRITER-1] stopping (signal)
    | 2026-10-27T12:19:20.339-07:00 [WRITER-2] stopping (signal)

  sent order (index: entity seq -> partition):
     0: RCL-2026-66434 seq=1 -> p0 (work 445 ms)
     1: RCL-2026-61335 seq=1 -> p1 (work 144 ms)
     2: RCL-2026-59801 seq=1 -> p0 (work 263 ms)
     3: RCL-2026-59801 seq=2 -> p1 (work 344 ms)
     4: RCL-2026-74265 seq=1 -> p0 (work 347 ms)
     5: RCL-2026-59801 seq=3 -> p1 (work 353 ms)
     6: RCL-2026-66434 seq=2 -> p0 (work 375 ms)
     7: RCL-2026-59801 seq=4 -> p1 (work 491 ms)
     8: RCL-2026-66434 seq=3 -> p0 (work 126 ms)
     9: RCL-2026-61335 seq=2 -> p1 (work 193 ms)
    10: RCL-2026-61335 seq=3 -> p0 (work 501 ms)
    11: RCL-2026-59801 seq=5 -> p1 (work 158 ms)
    12: RCL-2026-59801 seq=6 -> p0 (work 173 ms)
    13: RCL-2026-61335 seq=4 -> p1 (work 317 ms)
    14: RCL-2026-59801 seq=7 -> p0 (work 330 ms)
    15: RCL-2026-66434 seq=4 -> p1 (work 291 ms)
    16: RCL-2026-59801 seq=8 -> p0 (work 340 ms)
    17: RCL-2026-61335 seq=5 -> p1 (work 586 ms)
    18: RCL-2026-66434 seq=5 -> p0 (work 411 ms)
    19: RCL-2026-61335 seq=6 -> p1 (work 344 ms)

  processed order per entity:
    RCL-2026-59801: seq [2, 1, 3, 4, 5, 6, 7, 8]  partitions=[0, 1] writers=['writer-1', 'writer-2']  OUT OF ORDER
    RCL-2026-66434: seq [1, 2, 3, 4, 5]  partitions=[0, 1] writers=['writer-1', 'writer-2']  IN ORDER
    RCL-2026-74265: seq [1]  partitions=[0] writers=['writer-1']  IN ORDER
    RCL-2026-61335: seq [1, 2, 4, 3, 5, 6]  partitions=[0, 1] writers=['writer-1', 'writer-2']  OUT OF ORDER
2026-10-27T12:19:20.376-07:00 [PART3] RESULT unkeyed: processed=20 entities_out_of_order=2 inversions=2

================================================================================================
2026-10-27T12:19:20.376-07:00  ORDERING — keyed run: 20 messages (VERIFY_SEED=267875) through s7875_tasks (2 partitions), 2 Writers in one consumer group
================================================================================================
2026-10-27T12:19:20.526-07:00 [PART3] started writer-1 (pid 37449) group=s7875_writers_p3_ordering_keyed_121913 HW8_ORDER_LOG=/Users/atharvamundada/Desktop/data260-7875/reports/hw08/raw/part3_ordering_keyed.j
sonl  log=part3_ordering_keyed_writer-1.log
    | ===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | writer (writer-1) =====
    | 2026-10-27T12:19:20.646-07:00 [WRITER-1] in=s7875_tasks group=s7875_writers_p3_ordering_keyed_121913 idempotency=True commit_mode=after max_retries=3 work_delay_s=0.0 dead_letter=s7875_dead_
    | 2026-10-27T12:19:20.667-07:00 [WRITER-1] ASSIGNED partitions [0, 1]
    | 2026-10-27T12:19:20.669-07:00 [WRITER-1] writer is ready and listening on s7875_tasks
2026-10-27T12:19:20.730-07:00 [PART3] started writer-2 (pid 37452) group=s7875_writers_p3_ordering_keyed_121913 HW8_ORDER_LOG=/Users/atharvamundada/Desktop/data260-7875/reports/hw08/raw/part3_ordering_keyed.j
sonl  log=part3_ordering_keyed_writer-2.log
    | ===== Pinal Pawar | SID4 7875 | PREFIX s7875 | PORT_BASE 8675 | writer (writer-2) =====
    | 2026-10-27T12:19:20.832-07:00 [WRITER-2] in=s7875_tasks group=s7875_writers_p3_ordering_keyed_121913 idempotency=True commit_mode=after max_retries=3 work_delay_s=0.0 dead_letter=s7875_dead_
    | Heartbeat failed for group s7875_writers_p3_ordering_keyed_121913 because it is rebalancing
    | 2026-10-27T12:19:23.670-07:00 [WRITER-1] REVOKED partitions [0, 1]
    | 2026-10-27T12:19:23.678-07:00 [WRITER-1] ASSIGNED partitions [0]
    | 2026-10-27T12:19:23.679-07:00 [WRITER-2] ASSIGNED partitions [1]
    | 2026-10-27T12:19:23.683-07:00 [WRITER-2] writer is ready and listening on s7875_tasks
2026-10-27T12:19:23.741-07:00 [PART3] assignment: writer-1 -> [0]  writer-2 -> [1]
    | 2026-10-27T12:19:23.756-07:00 [WRITER-2] RECEIVE         corr=ord-keyed-121913-00 topic=s7875_tasks p=1 off=12 key=ord-keyed-121913-00 {"kind": "order_test", "run": "keyed", "message_id": "o
    | 2026-10-27T12:19:23.756-07:00 [WRITER-2] PROCESS-START   corr=ord-keyed-121913-00 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:19:23.757-07:00 [WRITER-1] RECEIVE         corr=ord-keyed-121913-01 topic=s7875_tasks p=0 off=20 key=ord-keyed-121913-01 {"kind": "order_test", "run": "keyed", "message_id": "o
    | 2026-10-27T12:19:23.758-07:00 [WRITER-1] PROCESS-START   corr=ord-keyed-121913-01 attempt=1 work_delay_s=0.0
2026-10-27T12:19:23.810-07:00 [PART3] sent 20 messages (key=entity)
    | 2026-10-27T12:19:23.903-07:00 [WRITER-1] ORDER-PROCESSED corr=ord-keyed-121913-01 entity=RCL-2026-61335 seq=1 p=0 off=20
    | 2026-10-27T12:19:23.907-07:00 [WRITER-1] COMMIT          corr=ord-keyed-121913-01 p=0 next_offset=21
    | 2026-10-27T12:19:23.909-07:00 [WRITER-1] RECEIVE         corr=ord-keyed-121913-09 topic=s7875_tasks p=0 off=21 key=ord-keyed-121913-09 {"kind": "order_test", "run": "keyed", "message_id": "o
    | 2026-10-27T12:19:23.909-07:00 [WRITER-1] PROCESS-START   corr=ord-keyed-121913-09 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:19:24.103-07:00 [WRITER-1] ORDER-PROCESSED corr=ord-keyed-121913-09 entity=RCL-2026-61335 seq=2 p=0 off=21
    | 2026-10-27T12:19:24.107-07:00 [WRITER-1] COMMIT          corr=ord-keyed-121913-09 p=0 next_offset=22
```

Unkeyed run: sent order (entity, seq, partition, work time) and processed order per entity - two notices OUT OF ORDER. Keyed run starting below it.

```
...op/data260-7875 — Pinal Pawar - SID 7875 — -zsh  ...p/data260-7875 — Pinal Pawar - SID 7875 — -zsh  .../data260-7875 — Pinal Pawar - SID 7875 — -zsh  .../data260-7875 — Pinal Pawar - SID 7875 — -zsh
    | 2026-10-27T12:19:26.726-07:00 [WRITER-2] RECEIVE         corr=ord-keyed-121913-12 topic=s7875_tasks p=1 off=21 key=ord-keyed-121913-12 {"kind": "order_test", "run": "keyed", "message_id": "o
    | 2026-10-27T12:19:26.726-07:00 [WRITER-2] PROCESS-START   corr=ord-keyed-121913-12 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:19:26.901-07:00 [WRITER-2] ORDER-PROCESSED corr=ord-keyed-121913-12 entity=RCL-2026-59801 seq=6 p=1 off=21
    | 2026-10-27T12:19:26.906-07:00 [WRITER-2] COMMIT          corr=ord-keyed-121913-12 p=1 next_offset=22
    | 2026-10-27T12:19:26.906-07:00 [WRITER-2] RECEIVE         corr=ord-keyed-121913-14 topic=s7875_tasks p=1 off=22 key=ord-keyed-121913-14 {"kind": "order_test", "run": "keyed", "message_id": "o
    | 2026-10-27T12:19:26.907-07:00 [WRITER-2] PROCESS-START   corr=ord-keyed-121913-14 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:19:27.238-07:00 [WRITER-2] ORDER-PROCESSED corr=ord-keyed-121913-14 entity=RCL-2026-59801 seq=7 p=1 off=22
    | 2026-10-27T12:19:27.245-07:00 [WRITER-2] COMMIT          corr=ord-keyed-121913-14 p=1 next_offset=23
    | 2026-10-27T12:19:27.245-07:00 [WRITER-2] RECEIVE         corr=ord-keyed-121913-15 topic=s7875_tasks p=1 off=23 key=ord-keyed-121913-15 {"kind": "order_test", "run": "keyed", "message_id": "o
    | 2026-10-27T12:19:27.245-07:00 [WRITER-2] PROCESS-START   corr=ord-keyed-121913-15 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:19:27.537-07:00 [WRITER-2] ORDER-PROCESSED corr=ord-keyed-121913-15 entity=RCL-2026-66434 seq=4 p=1 off=23
    | 2026-10-27T12:19:27.545-07:00 [WRITER-2] COMMIT          corr=ord-keyed-121913-15 p=1 next_offset=24
    | 2026-10-27T12:19:27.545-07:00 [WRITER-2] RECEIVE         corr=ord-keyed-121913-16 topic=s7875_tasks p=1 off=24 key=ord-keyed-121913-16 {"kind": "order_test", "run": "keyed", "message_id": "o
    | 2026-10-27T12:19:27.545-07:00 [WRITER-2] PROCESS-START   corr=ord-keyed-121913-16 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:19:27.887-07:00 [WRITER-2] ORDER-PROCESSED corr=ord-keyed-121913-16 entity=RCL-2026-59801 seq=8 p=1 off=24
    | 2026-10-27T12:19:27.896-07:00 [WRITER-2] COMMIT          corr=ord-keyed-121913-16 p=1 next_offset=25
    | 2026-10-27T12:19:27.896-07:00 [WRITER-2] RECEIVE         corr=ord-keyed-121913-18 topic=s7875_tasks p=1 off=25 key=ord-keyed-121913-18 {"kind": "order_test", "run": "keyed", "message_id": "o
    | 2026-10-27T12:19:27.896-07:00 [WRITER-2] PROCESS-START   corr=ord-keyed-121913-18 attempt=1 work_delay_s=0.0
    | 2026-10-27T12:19:28.309-07:00 [WRITER-2] ORDER-PROCESSED corr=ord-keyed-121913-18 entity=RCL-2026-66434 seq=5 p=1 off=25
    | 2026-10-27T12:19:28.318-07:00 [WRITER-2] COMMIT          corr=ord-keyed-121913-18 p=1 next_offset=26
    | 2026-10-27T12:19:28.324-07:00 [WRITER-1] stopping (signal)
    | 2026-10-27T12:19:28.396-07:00 [WRITER-2] stopping (signal)

  sent order (index: entity seq -> partition):
     0: RCL-2026-66434 seq=1 -> p1 (work 445 ms)
     1: RCL-2026-61335 seq=1 -> p0 (work 144 ms)
     2: RCL-2026-59801 seq=1 -> p1 (work 263 ms)
     3: RCL-2026-59801 seq=2 -> p1 (work 344 ms)
     4: RCL-2026-74265 seq=1 -> p1 (work 347 ms)
     5: RCL-2026-59801 seq=3 -> p1 (work 353 ms)
     6: RCL-2026-66434 seq=2 -> p1 (work 375 ms)
     7: RCL-2026-59801 seq=4 -> p1 (work 491 ms)
     8: RCL-2026-66434 seq=3 -> p1 (work 126 ms)
     9: RCL-2026-61335 seq=2 -> p0 (work 193 ms)
    10: RCL-2026-61335 seq=3 -> p0 (work 501 ms)
    11: RCL-2026-59801 seq=5 -> p1 (work 158 ms)
    12: RCL-2026-59801 seq=6 -> p1 (work 173 ms)
    13: RCL-2026-61335 seq=4 -> p0 (work 317 ms)
    14: RCL-2026-59801 seq=7 -> p1 (work 330 ms)
    15: RCL-2026-66434 seq=4 -> p1 (work 291 ms)
    16: RCL-2026-59801 seq=8 -> p1 (work 340 ms)
    17: RCL-2026-61335 seq=5 -> p0 (work 586 ms)
    18: RCL-2026-66434 seq=5 -> p1 (work 411 ms)
    19: RCL-2026-61335 seq=6 -> p0 (work 344 ms)

  processed order per entity:
    RCL-2026-59801: seq [1, 2, 3, 4, 5, 6, 7, 8]  partitions=[1] writers=['writer-2']  IN ORDER
    RCL-2026-66434: seq [1, 2, 3, 4, 5]  partitions=[1] writers=['writer-2']  IN ORDER
    RCL-2026-74265: seq [1]  partitions=[1] writers=['writer-2']  IN ORDER
    RCL-2026-61335: seq [1, 2, 3, 4, 5, 6]  partitions=[0] writers=['writer-1']  IN ORDER
2026-10-27T12:19:28.437-07:00 [PART3] RESULT keyed: processed=20 entities_out_of_order=0 inversions=0

================================================================================================
2026-10-27T12:19:28.439-07:00  FAILURE TABLE
================================================================================================
| Failure scenario | Before fix | After fix |
|---|---|---|
| Duplicate delivery | same task sent twice -> 2 model calls, 2 drafts (extra side effect) | 1 model call, 1 draft, 1 DUPLICATE-SKIP |
| Consumer killed mid-work | offset committed on receipt (committed=6 > msg offset 5); after restart redelivered=False -> LOST (never completed) | commit after send (committed=1 at kill); redelivered=True ->
completed after restart |
| Poison message | Writer crashed 3x on the same offset; good message processed=False (blocked) | 3 attempts -> s7875_dead_letter (1 record); good message processed=True |
| Two partitions/two consumers | no key: 2/4 notices processed out of order (2 inversions) | key=notice_code: 0/4 out of order; each notice on one partition |
(data260) Pinal Pawar ~/Desktop/data260-7875 %
```

Keyed run: every notice on one partition and one Writer, all IN ORDER; final failure table.

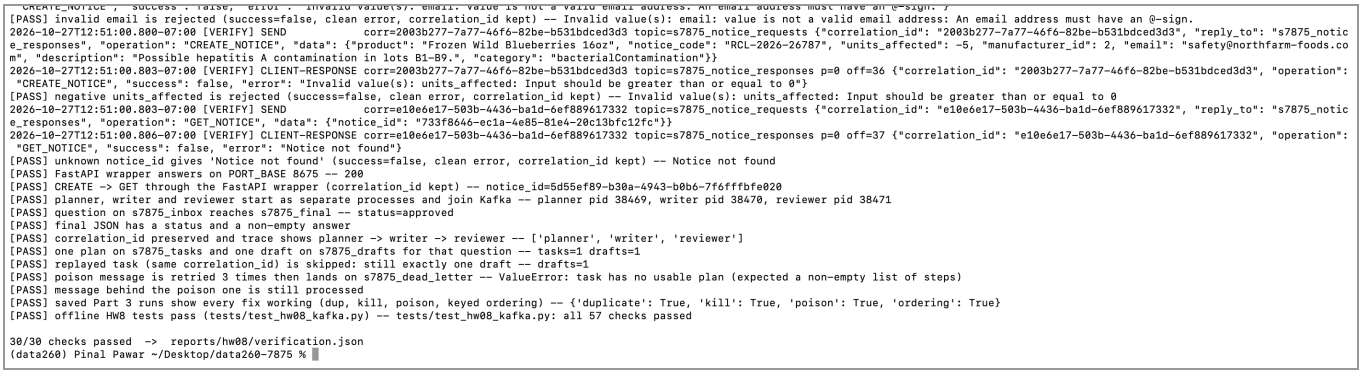
Data: raw/part3_ordering_unkeyed.csv / _keyed.csv (processed rank, time, writer, partition, offset, key, entity, seq), .jsonl, and raw/part3_ordering.json.

3.6 HW2 in-process graph vs. this Kafka design

	HW2 graph (function calls, one process)	HW8 (separate processes over Kafka)
Benefit 1: independent scaling	One process; to go faster the whole graph has to be copied.	Add a Writer process to the group and Kafka splits s7875_tasks between them (two Writers, one partition each in 3.5).
Benefit 2: decoupled failure	If a node raised, the call stack unwound and the run was lost.	A dead Writer stops only itself; its unfinished message stays in Kafka and is picked up on restart (3.3). Planner and Reviewer keep running.
New responsibility 1: duplicates	Each step ran exactly once - a function was called once.	Delivery is at-least-once, so the same message can arrive twice; idempotency keys are now our job (3.2).
New responsibility 2: ordering and offsets	Steps ran in program order; there was nothing to commit.	Order holds only within a partition, so related messages need a key (3.5); and we decide when an offset is committed, which decides lost vs. redelivered work (3.3), plus poison messages that would block a partition (3.4).

Self-check (verification.json)

`make verify-hw08` (`scripts/verify_hw08.py`) runs a smoke test on the committed code and writes `reports/hw08/verification.json` with the homework number, SID4, commit hash, model/configuration, SEED / VERIFY_SEED, and pass/fail for each check. It tests behaviour only (success flags, ids echoed, counts, messages arriving on the right topic), starts the agents on new consumer groups with a temporary state folder, and never changes application code. Result: **30/30 checks passed** - config and topics, the three containers and their log lines, CREATE → GET over Kafka and through FastAPI, six error cases, a replayed CREATE creating no second notice, the Part 2 journey with qwen3:1.7b, duplicate skip, poison → dead letter with a later message still processed, the saved Part 3 results, and 57 offline tests.



make verify-hw08 on commit 58c8954: 30/30 checks passed → reports/hw08/verification.json

AI Use

Same text as reports/hw08/AI_USE.md.

1. What I used an AI assistant for, and what I did myself

I used Claude to read the assignment and the two class demos (`DATA236_DEMO8`, `multi-agent-demo`) with me, and to understand worker, FastAPI wrapper, Kafka client, shared agent loop with idempotency, commit modes, bounded retries and dead-letter, `homework_demo.py`, the three agents and `send_question.py`, `docker-compose.kafka.yml`, the topic-creation script, the Part 3 failure harness, `verify_hw08.py` and the offline tests. It also drafted the report around my screenshots.

Claude could not run Kafka itself (no Docker in its workspace), so I tested the code against an in-memory stand-in broker. Everything reported here I ran myself on a MacBook Air (Apple M2, 16 GB) with Docker Desktop, real Kafka 3.9.2 and qwen3:1.7b in Ollama: the containers, topic creation, both demo runs, both Part 2 runs in four terminals, the four failure tests and the self-check. I took every screenshot and read the outputs before accepting them. The decisions about what to keep, re-run or fix were mine (for example re-running Part 2 after the safety fix below, keeping run 1 as evidence, and keeping `notice` in the topic names).

2. One AI-produced output that was wrong/unsuitable

The Planner/Writer/Reviewer prompts the assistant first wrote had no food-safety rules. In my first real Part 2 run (`raw/part2_journey_d5f6857e.json`), qwen3:1.7b planned a step "Determine if the spinach you ate is still safe to consume", wrote **"If the spinach you ate is from the same batch and within the expiration window, it may still be safe"**, and the Reviewer **approved** it. The Kafka side was perfect (same correlation_id on all four topics, every offset committed), but the answer was wrong: a recalled product is never safe to eat, whatever its date.

A smaller one: the first topic-creation script stopped after describing one topic, because `set -o pipefail` plus `| head -1` turned the pipe closing early into an error.

3. How I detected the problem

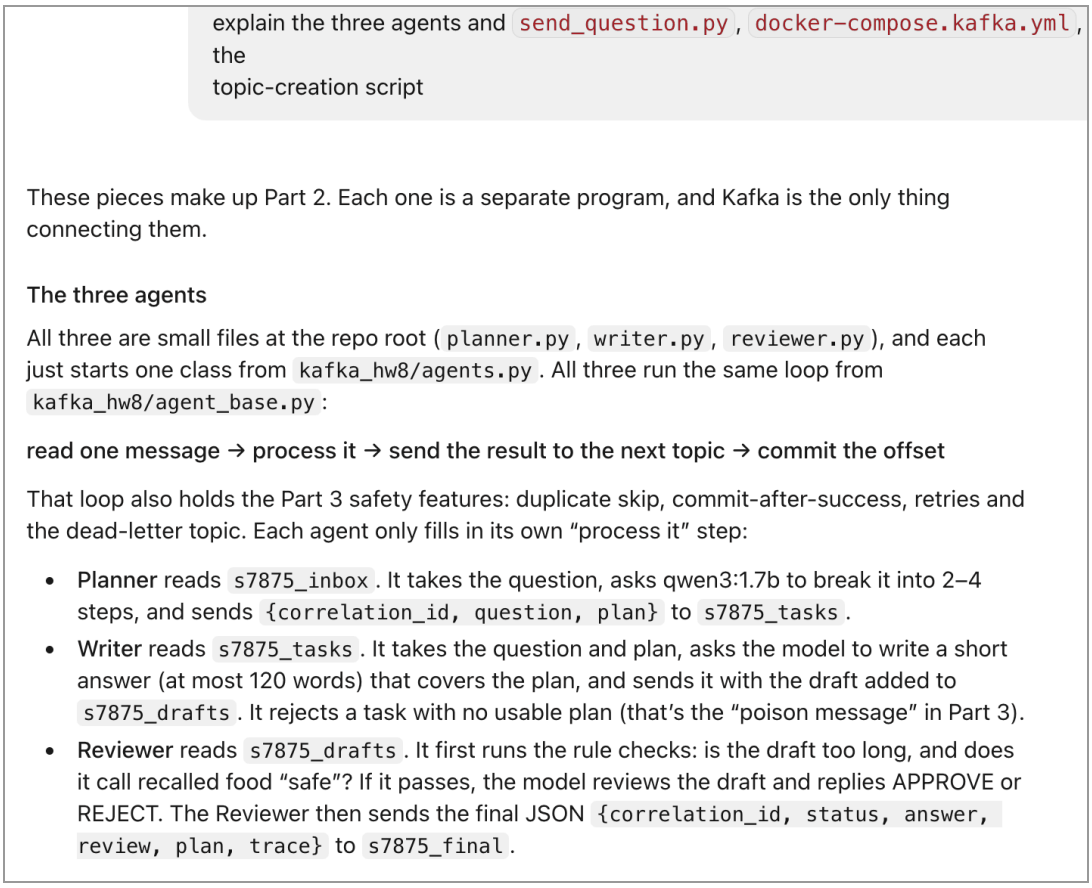
By reading the final JSON in my run-1 screenshot rather than only checking that `status` was present. The structural checks (status, answer, correlation_id, trace) all passed, which is exactly why they could not catch it: they test the pipeline, not whether the advice is right. The topic-script bug showed in my screenshot: only `s7875_notice_requests` appeared under `--describe`.

4. What I changed and why it works now

- All three agents now get explicit recall-safety rules (never eat a recalled product, throw it away or return it, clean surfaces, watch for symptoms and call a doctor, sooner for high-risk people).
- The Reviewer runs a **rule check before the model**: a draft with phrases like "may still be safe" or "safe to eat" is rejected outright, so a small model's review is no longer the only safeguard. Negated forms ("it is **not** safe to eat") are allowed.

- Offline tests now include the exact sentence from run 1 and check it is rejected (`tests/test_hw08_kafka.py`, 57/57 pass).

Re-run (run 2, `raw/part2_journey_b14cad9d.json`): "Stop using the recalled bagged organic baby spinach and throw it away or return it to the store ... Do not eat the product, regardless of its appearance or smell", approved, 16.2 s end to end. The topic script now describes all seven topics and `s7875_tasks` shows `PartitionCount: 2`.



explain the three agents and `send_question.py`, `docker-compose.kafka.yml`, the
topic-creation script

These pieces make up Part 2. Each one is a separate program, and Kafka is the only thing connecting them.

The three agents

All three are small files at the repo root (`planner.py`, `writer.py`, `reviewer.py`), and each just starts one class from `kafka_hw8/agents.py`. All three run the same loop from `kafka_hw8/agent_base.py`:

read one message → process it → send the result to the next topic → commit the offset

That loop also holds the Part 3 safety features: duplicate skip, commit-after-success, retries and the dead-letter topic. Each agent only fills in its own "process it" step:

- **Planner** reads `s7875_inbox`. It takes the question, asks qwen3:1.7b to break it into 2–4 steps, and sends `{correlation_id, question, plan}` to `s7875_tasks`.
- **Writer** reads `s7875_tasks`. It takes the question and plan, asks the model to write a short answer (at most 120 words) that covers the plan, and sends it with the draft added to `s7875_drafts`. It rejects a task with no usable plan (that's the "poison message" in Part 3).
- **Reviewer** reads `s7875_drafts`. It first runs the rule checks: is the draft too long, and does it call recalled food "safe"? If it passes, the model reviews the draft and replies APPROVE or REJECT. The Reviewer then sends the final JSON `{correlation_id, status, answer, review, plan, trace}` to `s7875_final`.

Proof of AI use: my Claude conversation during HW8, asking it to explain the three agents, send_question.py, docker-compose.kafka.yml and the topic-creation script.